

Course Notes in C++ Design & Analysis of Algorithms

*Twenty-eight topics, sixty-six boards, one page each.
The whole semester in the order it is taught.*

These are the class boards, typeset. Every topic gets one page: four ideas, the code for each, a contest note, and four lines worth remembering. Nothing has been padded out. If you sat through the lecture, this is what was on the screen; if you missed it, this is the shortest honest version.

STL POWER-UPS

BINARY TREES

BSTs

GREEDY

DYNAMIC PROGRAMMING

GRAPHS

BACKTRACKING

TWO POINTERS

WHAT IS IN HERE

9 modules 28 chapters 66 boards 264 panels

One board to a page. Four ideas, four code blocks, four lines to remember.

CONTENTS

What is inside

Nine modules, twenty-eight chapters, one page per board.

MODULE ONE · STL POWER-UPS AND FUNDAMENTALS

1	Lambda Functions	5 boards	5
2	STL Containers	3 boards	10
3	STL Algorithms	3 boards	13
4	Core Revision	3 boards	16

MODULE TWO · BINARY TREES: TRAVERSALS AND VIEWS

5	Tree Traversals	3 boards	20
6	Tree Properties	2 boards	23
7	Tree Views	2 boards	25

MODULE THREE · BINARY TREES: PATH SUMS, LCA AND OPS

8	Tree Structure Operations	2 boards	28
9	Tree Path Sums	2 boards	30
10	LCA and Construction	3 boards	32

MODULE FOUR · BINARY SEARCH TREES

11	BST Basics	2 boards	36
12	BST Operations	2 boards	38
13	BST Queries	2 boards	40

MODULE FIVE · GREEDY ALGORITHMS AND PATTERNS

14	Greedy on Intervals	2 boards	43
15	Greedy Scheduling	2 boards	45
16	Greedy on Arrays	2 boards	47

MODULE SIX · DYNAMIC PROGRAMMING: 1D, LIS AND GRIDS

17	DP Foundations	2 boards	50
18	LIS and LCS	2 boards	52

19	Grids and Subset Sum 2 boards	54
----	-------------------------------	----

MODULE SEVEN · DYNAMIC PROGRAMMING: ADVANCED AND PARTITION

20	Knapsack 2 boards	57
21	String DP 2 boards	59
22	Partition DP 2 boards	61

MODULE EIGHT · GRAPH ALGORITHMS AND SHORTEST PATHS

23	Graph Traversals 3 boards	64
24	Shortest Paths 2 boards	67
25	Minimum Spanning Trees 2 boards	69

MODULE NINE · TREE DP, BACKTRACKING AND CAPSTONE

26	Tree DP 2 boards	72
27	Backtracking 3 boards	74
28	Sliding Window and Two Pointers 2 boards	77

HOW TO READ THIS BOOK

Every chapter is one folder from the course repository, and every page is one board from class. The four panels on a page are four ideas, in the order they were taught.

The **CP** line under each code block is the contest note: why the idea matters when the clock is running. The panel at the foot of each page is the four lines to carry into an exam.

Code here is the short form from the board. The full runnable files sit next to these notes in the folder, and all sixty-six of them compile under `g++ -std=c++17`.

MODULE ONE

STL Power-Ups and Fundamentals

The tools you will use in every problem after this. Lambdas for short logic, the containers worth knowing, the four algorithms that do most of the work, and a revision pass over recursion, stacks and linked lists.

1	Lambda Functions	5 boards
	Basics, Capture Lists, STL Usage, Recursive Lambdas, Quick Revision	
2	STL Containers	3 boards
	Vector Deque, Map Set, Quick Revision	
3	STL Algorithms	3 boards
	Sort Comparators, Binary Search, Priority Queue	
4	Core Revision	3 boards
	Recursion, Stack Dry Run, Linked List	

4 chapters · 14 boards · folders 01 to 04

CHAPTER 1

5 BOARDS · 01_Lambdas

Lambda Functions

A lambda is a function you write where you need it, without giving it a name. Keep it in `auto`, because its type is one the compiler invents and you cannot spell.

Three shapes cover almost everything you will write this semester: a comparator with no capture, a feasibility check with `[&]`, and a recursive traversal that takes itself as a parameter. The five boards below build up to those three.

BOARD 1

Basics

01 What is a Lambda?

A function written inline, where you need it. It has no name. Keep it in `auto`.

```
auto add = [](int a, int b) {
    return a + b;
};
cout << add(2, 3);    // 5
```

CP Small helpers inside `main()`.

02 Basic Syntax

`[capture] (params) -> ret { body }` Only `[]` and `{ }` are compulsory.

```
[](int x) { return x * x; };
[] { cout << "hi"; };    // () dropped
```

CP One shape, used everywhere.

03 Parameters

Same as normal function parameters. Use `&` for big containers.

```
auto sum = [](vector<int> &v) {
    long long s = 0;
    for (int x : v) s += x;
    return s;
};
```

CP Pass vectors by reference, never by copy.

04 Return Value

Type is deduced on its own. Write `-> T` when you want to force it.

```
auto half = [](int x) { return x / 2; };
auto sq = [](int n) -> long long {
    return 1LL * n * n;
};
```

CP `-> long long` saves you from overflow.

REMEMBER

- ▶ Lambda = function with no name.
- ▶ Keep it in `auto`.
- ▶ `[]` must be there, `()` is optional.
- ▶ Add `-> T` if the type matters.

BOARD 2

Capture Lists

01 Capture List

`[]` says what the lambda can see from outside. An empty `[]` sees nothing.

```
int n = 5;
auto f = [](int x) { return x + n; }; // error
auto g = [n](int x) { return x + n; }; // ok
```

CP Forgetting to capture is the usual compile error.

02 `[]` and `[=]`

`[]` takes nothing. `[=]` takes copies. A copy is frozen at that moment.

```
int n = 3;
auto f = [=](int x) { return x * n; };
n = 100;
cout << f(2); // 6
```

CP `[=]` copies big vectors, so it can be slow.

03 `[&]`

Uses the real variables, not copies. Changes inside are seen outside.

```
int cnt = 0;
auto hit = [&]() { cnt++; };
hit(); hit();
cout << cnt; // 2
```

CP Use `[&]` by default in contests.

04 `[x]` and `[&x]`

Pick one variable: `[x]` copy, `[&x]` reference. You can mix them: `[x, &y]`

```
int a = 1, b = 1;
auto f = [a, &b]() { b += a; };
f();
cout << b; // 2
```

CP Change one variable, keep the rest frozen.

REMEMBER

- ▶ `[]` none, `[=]` copy, `[&]` reference.
- ▶ `[x]` copy of `x`, `[&x]` the real `x`.
- ▶ In CP, use `[&]`.

BOARD 3

STL Usage

01 **sort() comparator**

Return true if a should come first.

```
sort(v.begin(), v.end(), [](auto a, auto b) {
    return a.second < b.second;
});
```

CP Sort by any field, no extra struct needed.

02 **priority_queue comparator**

Logic is reversed: true means lower priority. The type needs decltype.

```
auto cmp = [](int a, int b) { return a > b; };
priority_queue<int, vector<int>, decltype(cmp)> pq(cmp);
pq.push(5); pq.push(1);
cout << pq.top();    // 1
```

CP Quick min-heap for greedy problems.

03 **Binary Search check()**

Put the test inside a lambda. [&] lets it read the input directly.

```
auto ok = [&](long long m) {
    return m * m <= x;
};
while (lo < hi) {
    long long mid = (lo + hi + 1) / 2;
    if (ok(mid)) lo = mid; else hi = mid - 1;
}
```

CP The binary-search-on-answer pattern.

04 **Generic Lambda (auto)**

auto in the params = works for many types. C++14 and later.

```
auto mx = [](auto a, auto b) {
    return a > b ? a : b;
};
cout << mx(3, 7);
```

CP One lambda for int, long long, pair.

REMEMBER

- ▶ sort: true means a comes first.
- ▶ priority_queue: logic flipped, pass cmp to it.
- ▶ check() with [&] keeps the code short.
- ▶ auto params = any type.

BOARD 4

Recursive Lambdas

01 Why recursion is tricky

A lambda has no name, so it cannot call itself.

```
auto f = [](int n) {
    return n * f(n - 1);    // error
};
```

CP Still needed: DFS written inside `main()`.

02 C++17 - pass self

Take the lambda itself as the first parameter. Then call `self(self, ...)`.

```
auto fact = [](auto &self, int n) -> long long {
    if (n <= 1) return 1;
    return n * self(self, n - 1);
};
cout << fact(fact, 5);    // 120
```

CP The usual Codeforces trick.

03 C++20 - `std::function`

Give it a name first, then call that name. Needs `<functional>`, and is a bit slower.

```
function<long long(int)> fact = [](int n) {
    if (n <= 1) return 1LL;
    return n * fact(n - 1);
};
cout << fact(5);          // 120
```

CP Easy to read. Avoid in heavy recursion.

04 DFS

Capture the graph and visited with `[&]`. Return type must be written: `-> void`.

```
auto dfs = [&](auto &self, int u) -> void {
    vis[u] = 1;
    for (int v : g[u]) if (!vis[v]) self(self, v);
};
dfs(dfs, 0);
```

CP Components, tree traversal, subtree sizes.

REMEMBER

- ▶ A lambda cannot see its own name.
- ▶ C++20: `function<ret(args)>`.
- ▶ C++17: `self(self, n)`.
- ▶ Always write `→ void` or `→ long long`.

BOARD 5

Quick Revision

01 When to use Lambdas

Short logic, needed in one place only.

```
sort(v.begin(), v.end(), [](auto a, auto b) {
    return a > b;
});
```

CP Comparators, `check()`, DFS.

02 When NOT to use Lambdas

Long logic, or logic reused many times. Write a normal function instead.

```
bool cmp(Node a, Node b) { return a.w < b.w; }
sort(e.begin(), e.end(), cmp);
```

CP Long lambdas are hard to debug.

03 Capture cheat sheet

All five forms in one place.

[]	nothing	
[=]	copy of all	
[&]	reference to all	<-- CP default
[x]	copy of x	
[&x]	reference to x	

CP Start with [&], narrow it only if needed.

04 The 3 patterns to remember

Comparator, `check()`, recursion.

```
sort(a.begin(), a.end(), [](int x, int y) { return x > y; });
auto ok = [&](long long m) { return true; };
auto dfs = [&](auto &self, int u) -> void { self(self, u); };
```

CP These three cover almost every contest use.

REMEMBER

- ▶ `auto f = [capture](params) → ret { body };`
- ▶ Recursion: pass self, write the return type.
- ▶ Use [&] first.
- ▶ Long logic → normal function.

CHAPTER 2

3 BOARDS · 02_STL_Containers

STL Containers

Pick the container from the operation you need most, not from habit. `vector` for almost everything, `map` and `set` when order matters, the unordered versions when it does not.

The two things that actually bite: `operator[]` inserts on a read, and iterators go stale after a reallocation.

BOARD 1

Vector Deque

01 Growing a `vector`

`push_back` copies, `emplace_back` builds in place. Both are amortised $O(1)$.

```
vector<int> v;
v.push_back(1);
v.emplace_back(2);
cout << v.size();    // 2
```

CP `emplace_back` for structs and pairs.

02 `reserve` vs `resize`

`reserve` sets capacity only, size stays 0. `resize` actually creates elements.

```
v.reserve(100);
cout << v.size() << " " << v.capacity();    // 2 100
v.resize(5);
cout << v.size();    // 5
```

CP `reserve` before a big loop kills reallocation.

03 `insert` and `erase`

Both are $O(n)$: everything after shifts. `erase-remove` is the way to drop by value.

```
v.erase(v.begin() + 1);
v.erase(remove(v.begin(), v.end(), 3), v.end());
```

CP Never erase inside a plain index loop.

04 `deque`

Push and pop at both ends in $O(1)$. No contiguous memory guarantee.

```
deque<int> d = {2, 3};
d.push_front(1);
d.push_back(4);
cout << d.front() << d.back();    // 1 4
```

CP `deque` for 0-1 BFS and sliding windows.

REMEMBER

- ▶ `push_back` amortised $O(1)$, `insert/erase` $O(n)$.
- ▶ `reserve` = capacity, `resize` = size.
- ▶ 2D: `vector<vector<int>> g(n, vector<int>(m));`
- ▶ `deque` when you need both ends.

BOARD 2

Map Set

01 map is sorted

Keys stay in sorted order. All ops $O(\log n)$. Iteration is always ascending.

```
map<string, int> m;
m["b"] = 2;
m["a"] = 1;
for (auto [k, v] : m) cout << k;    // a b
```

CP Frequency counting in one line: `m[x]++`.

02 operator[] inserts

Reading a missing key creates it with 0. Use `count` or `find` to only look.

```
cout << m["zz"];           // 0, and zz now exists
cout << m.size();          // 3
if (m.count("a")) cout << "yes";
```

CP This silently grows the map. Use `count`.

03 set

Unique, sorted, $O(\log n)$. `multiset` allows copies. Use the member `lower_bound`, not `std::`.

```
set<int> s = {10, 20, 30};
cout << *s.lower_bound(15);    // 20
cout << *prev(s.lower_bound(25)); // 20
```

CP `std::lower_bound` on a set is $O(n)$. Wrong tool.

04 unordered variants

Hashing, $O(1)$ average, no ordering. Worst case $O(n)$ on adversarial keys.

```
unordered_map<int, int> f;
f[5]++;
cout << f[5];                // 1
```

CP Need sorted order or `lower_bound`? Use map.

REMEMBER

- ▶ map/set sorted $O(\log n)$, unordered $O(1)$ avg.
- ▶ Member `lower_bound` on set, never `std::`.
- ▶ `m[k]` inserts. `count`/`find` only look.
- ▶ `multiset` `erase(k)` drops all copies.

BOARD 3

Quick Revision

01 Picking a container

Random access, both ends, sorted, or hashed. That one question picks it for you.

```
vector  random access, push_back
deque  O(1) at both ends
map/set sorted, lower_bound
unordered_* O(1) average
```

CP Default to `vector` until something forces a change.

02 The ops you actually use

Four per container covers almost everything.

```
v.push_back v.size sort(v.begin(), v.end())
m[k]++ m.count(k) m.find(k)
s.insert s.count s.lower_bound
```

CP Learn these cold, look up the rest.

03 Iterator invalidation

A `vector` reallocates and old pointers die. map and set nodes are stable.

```
int *p = &v[0];
v.push_back(9);           // p may now dangle
```

CP Never hold a `vector` pointer across `push_back`.

04 Complexity table

Know these before you pick.

```
vector  index O(1) insert/erase O(n)
deque  index O(1) ends O(1)
map/set all O(log n)
```

CP An $O(n)$ erase inside a loop is $O(n^2)$.

REMEMBER

- ▶ `vector` by default.
- ▶ `deque` for both ends.
- ▶ map when you need order, unordered for speed.
- ▶ Vector reallocation invalidates pointers.

CHAPTER 3

3 BOARDS · 03_STL_Algorithms

STL Algorithms

Four algorithms carry most contest code: `sort` with a comparator, the two binary search bounds, `priority_queue`, and binary search on the answer.

Watch the two direction traps. A comparator must be strictly less than, never less than or equal. A `priority_queue` comparator reads backwards, so `a > b` gives you a min-heap.

BOARD 1

Sort Comparators

01 `sort`

$O(n \log n)$. Comparator returns true if a goes first. Default is ascending.

```
sort(v.begin(), v.end());
sort(v.begin(), v.end(), greater<int>());
```

CP `greater<int>()` is the fastest descending sort.

02 `Lambda comparator`

Sort by any field with no struct needed.

```
sort(v.begin(), v.end(), [](auto a, auto b) {
    return a.second < b.second;
});
```

CP Sort pairs by second, or by any key you like.

03 `Tie-breaking`

Compare the next field only when equal.

```
sort(e.begin(), e.end(), [](Node a, Node b) {
    if (a.w != b.w) return a.w < b.w;
    return a.id < b.id;
});
```

CP `stable_sort` keeps the original order on ties.

04 `The <= bug`

A comparator must be strictly less. Using `<=` is undefined behaviour, not just wrong.

```
return a <= b;    // UB, can crash
return a < b;     // correct
```

CP Never write `<=` or `>=` inside a comparator.

REMEMBER

- ▶ Comparator: true means a comes first.
- ▶ `stable_sort` preserves ties.
- ▶ Must be strict: `<` not `<=`.
- ▶ `sort` is $O(n \log n)$.

BOARD 2

Binary Search

01 The two bounds

`lower_bound`: first $\geq x$. `upper_bound`: first $> x$. The array must already be sorted.

```
vector<int> v = {10, 20, 20, 20, 30};
cout << *lower_bound(v.begin(), v.end(), 20); // 20
cout << *upper_bound(v.begin(), v.end(), 20); // 30
```

02 Iterator to index

Subtract `begin()` to get a position.

```
int i = lower_bound(v.begin(), v.end(), 20)
      - v.begin();
cout << i; // 1
```

03 Counting a value

The gap between the bounds is the count.

```
cout << upper_bound(v.begin(), v.end(), 20)
      - lower_bound(v.begin(), v.end(), 20); // 3
```

CP `equal_range` returns both at once.

04 Search on the answer

Wrap the test in a lambda. It must be false..false then true..true.

```
auto ok = [&](long long m) { return m * m <= x; };
while (lo < hi) {
    long long mid = (lo + hi + 1) / 2;
    if (ok(mid)) lo = mid; else hi = mid - 1;
}
```

CP Largest-true needs +1 in mid or it hangs.

REMEMBER

- ▶ Array must be sorted first.
- ▶ Count = upper - lower.
- ▶ `lower_bound` $\geq x$, `upper_bound` $> x$.
- ▶ Largest true: $\text{mid} = (\text{lo} + \text{hi} + 1) / 2$.

BOARD 3

Priority Queue

01 Default is a max-heap

top is the largest. push and pop are $O(\log n)$, top is $O(1)$.

```
priority_queue<int> pq;
pq.push(5); pq.push(1); pq.push(9);
cout << pq.top();      // 9
```

CP No iteration. Only top, push, pop.

02 Min-heap

greater<int> flips it. Simplest form.

```
priority_queue<int, vector<int>, greater<int>> pq;
pq.push(5); pq.push(1);
cout << pq.top();      // 1
```

CP Memorise this line. Dijkstra needs it.

03 Lambda comparator

Logic is reversed: true means lower priority. The type needs decltype.

```
auto cmp = [](int a, int b) { return a > b; };
priority_queue<int, vector<int>,
    decltype(cmp)> pq(cmp);
```

CP Opposite of sort. This trips everyone up.

04 Heap of pairs

Compares first, then second. Put the key first.

```
priority_queue<pair<long long, int>,
    vector<pair<long long, int>>,
    greater<>> pq;
pq.push({dist, node});
```

CP The {dist, node} shape is Dijkstra's whole trick.

REMEMBER

- ▶ Default max-heap, greater<> for min-heap.
- ▶ push/pop $O(\log n)$, top $O(1)$.
- ▶ Comparator logic is flipped vs sort.
- ▶ Pairs compare first, then second.

CHAPTER 4

3 BOARDS · 04_Core_Revision

Core Revision

A pass over the three things the rest of the course assumes: recursion, the call stack, and pointer surgery on a linked list.

If the dry run on paper does not match what your code prints, the bug is in your model of the stack, not in the compiler.

BOARD 1

Recursion

01 Base case first

No base case means infinite recursion. Every call must move toward it.

```
long long fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}
```

CP Write the base case before the recursive line.

02 Branching blows up

Two calls per level means 2^n work. The same subproblem is recomputed.

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
// fib(30) = 2692537 calls
```

CP This is exactly what DP fixes.

03 Memoisation

Cache each answer once. $O(n)$ instead of $O(2^n)$.

```
vector<long long> memo(n + 1, -1);
auto f = [&](auto &self, int i) -> long long {
    if (i <= 1) return i;
    if (memo[i] != -1) return memo[i];
    return memo[i] = self(self, i-1) + self(self, i-2);
};
```

CP One array turns brute force into DP.

04 Recursion on arrays

Shrink the range, combine on the way back.

```
int sum(vector<int> &a, int i) {
    if (i == (int)a.size()) return 0;
    return a[i] + sum(a, i + 1);
}
```

CP Depth = n , so watch the stack on big n .

REMEMBER

- ▶ Base case, then recursive case.
- ▶ Branching recursion is exponential.
- ▶ Memo array collapses it to $O(n)$.
- ▶ Recursion depth costs stack memory.

BOARD 2

Stack Dry Run

01 Stack basics

Last in, first out. All ops $O(1)$. top on an empty stack is undefined.

```
stack<int> st;
st.push(1); st.push(2);
cout << st.top();      // 2
st.pop();
cout << st.size();     // 1
```

CP Always guard with `!st.empty()` before top.

02 Balanced brackets

Push openers, match on every closer. Empty at the end means balanced.

```
for (char c : s) {
    if (c == '(') st.push(c);
    else { if (st.empty()) return false; st.pop(); }
}
return st.empty();
```

CP Early empty is a fail, not just leftovers.

03 Monotonic stack

Keep the stack decreasing. Whatever pops out has just found its next greater element.

```
for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.top()] < a[i])
        { ans[st.top()] = a[i]; st.pop(); }
    st.push(i);
}
```

CP $O(n)$ total. Each index pushed and popped once.

04 Dry run on paper

Write the stack after every step. That is what exams ask you to show.

```
// a = 5 2 6
// i=0 st=[5]
// i=1 st=[5,2]
// i=2 pop 2 → 6, pop 5 → 6, st=[6]
```

CP ans = 6 6 -1. Leftovers get -1.

REMEMBER

- ▶ LIFO, all operations $O(1)$.
- ▶ Monotonic stack = next greater in $O(n)$.
- ▶ Check empty before top or pop.
- ▶ Anything left on the stack has no answer.

BOARD 3

Linked List

01 The node

A value and a pointer to the next node. `nullptr` marks the end.

```
struct Node {
    int val;
    Node *next;
    Node(int v) : val(v), next(nullptr) {}
};
```

CP Always initialise next to `nullptr`.

02 Insert and delete

Insert at head is $O(1)$. Deleting needs the node before the target.

```
Node *n = new Node(x);
n->next = head;
head = n;           // push front
```

CP Deleting the head is the case people forget.

03 Reverse iteratively

Three pointers. Flip one link per step.

```
Node *prev = nullptr, *cur = head, *nxt;
while (cur) {
    nxt = cur->next;
    cur->next = prev;
    prev = cur; cur = nxt;
}
```

CP Save next before you overwrite the link.

04 Slow and fast

Fast moves two, slow moves one. Slow lands on the middle. If they meet there is a cycle.

```
while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
    if (slow == fast) return true; // cycle
}
```

CP Floyd's trick. $O(n)$ time, $O(1)$ space.

REMEMBER

- ▶ Node = value + next pointer.
- ▶ Reverse needs prev, cur, next.
- ▶ Slow/fast finds middle and cycles.
- ▶ Head and empty list are the edge cases.

MODULE TWO

Binary Trees: Traversals and Views

Every tree problem is a traversal with something extra carried along. Learn the three orders, then the iterative forms, then what you can measure and see from outside the tree.

5	Tree Traversals	3 boards
	Recursive DFS, Iterative And BFS, Morris Traversal	

6	Tree Properties	2 boards
	Height And Depth, Diameter And Balanced	

7	Tree Views	2 boards
	Top And Bottom View, Vertical And Boundary	

3 chapters · 7 boards · folders 05 to 07

CHAPTER 5

3 BOARDS · 05_Tree_Traversals

Tree Traversals

One recursive shape, three orders. Where you put the visit line decides whether it is preorder, inorder or post-order.

Then the same three without recursion, level order with a queue, and Morris traversal for when you are asked to use $O(1)$ extra space.

BOARD 1

Recursive DFS

01 Three orders, one shape

Only the position of the visit changes. Left and right recursion stay put.

```
void pre(Node *r) {
    if (!r) return;
    cout << r->val;           // visit first
    pre(r->left); pre(r->right);
}
```

CP Base case is if (!r) return. Always.

02 Inorder

Visit between the two calls. On a BST this prints sorted order.

```
in(r->left);
cout << r->val;
in(r->right);
// 4 2 7 5 1 3 6 8
```

CP Inorder on a BST is the sorted check.

03 Postorder

Visit last, after both children. Use it when the parent needs child results.

```
post(r->left); post(r->right);
cout << r->val;
// 4 7 5 2 8 6 3 1
```

CP Every bottom-up tree DP is postorder.

04 Lambda version

Write the DFS inside main with self(self, node).

```
auto dfs = [&](auto &self, Node *r) -> void {
    if (!r) return;
    self(self, r->left);
    self(self, r->right);
};
dfs(dfs, root);
```

CP No global functions needed. Write $\rightarrow$ void.

REMEMBER

- ▶ Pre, in, post: only the visit moves.
- ▶ Inorder on BST gives sorted order.
- ▶ Base case: if (!r) return.
- ▶ Postorder for bottom-up results.

BOARD 2

Iterative And BFS

01 Iterative preorder

One stack, seeded with root. Push right before left so left comes off first.

```
while (!st.empty()) {
    Node *n = st.top(); st.pop();
    cout << n->val;
    if (n->right) st.push(n->right);
    if (n->left) st.push(n->left);
}
```

CP Push right first or the order flips.

02 Iterative inorder

Walk left pushing, then pop and go right.

```
while (cur || !st.empty()) {
    while (cur) { st.push(cur); cur = cur->left; }
    cur = st.top(); st.pop();
    cout << cur->val;
    cur = cur->right;
}
```

CP The pattern is: dive left, pop, turn right.

03 Level order BFS

A queue, not a stack. Visits by depth. Seed it with root.

```
while (!q.empty()) {
    Node *n = q.front(); q.pop();
    cout << n->val;
    if (n->left) q.push(n->left);
    if (n->right) q.push(n->right);
}
```

CP Queue = breadth. Stack = depth.

04 Level by level

Read the queue size first. That is exactly one level.

```
while (!q.empty()) {
    int sz = q.size();
    while (sz--) { /* one level */
        cout << " ";
    }
    cout << "\n";
}
```

CP The size trick is needed for views and widths.

REMEMBER

- ▶ Stack = DFS, queue = BFS.
- ▶ Inorder: dive left, pop, turn right.
- ▶ Preorder: push right before left.
- ▶ Level split: read q.size() first.

BOARD 3

Morris Traversal

01 Why Morris

Inorder in $O(1)$ space. No stack, no recursion. It borrows the null right pointers.

```
// normal inorder:  $O(h)$  stack  
// Morris:  $O(1)$  extra space
```

CP Asked when the interviewer says no extra space.

02 The thread

Find the inorder predecessor: rightmost node of the left subtree. Point it back at cur.

```
Node *p = cur->left;  
while (p->right && p->right != cur)  
    p = p->right;
```

CP The $p \rightarrow \text{right} \neq \text{cur}$ test finds an existing thread.

03 Build then break

No thread yet: create it and go left. Thread found: print, remove it, go right.

```
if (!p->right) { p->right = cur; cur = cur->left; }  
else { p->right = nullptr; cout << cur->val;  
      cur = cur->right; }
```

CP Each node is reached at most twice, so $O(n)$.

04 No left child

Print immediately and move right.

```
if (!cur->left) { cout << cur->val;  
               cur = cur->right; }
```

CP Tree ends up unchanged. Verify by rerunning inorder.

REMEMBER

- ▶ Inorder in $O(1)$ space.
- ▶ Thread the inorder predecessor to cur.
- ▶ Second visit: print, unthread, go right.
- ▶ Still $O(n)$: each node seen twice.

CHAPTER 6

2 BOARDS · 06_Tree_Properties

Tree Properties

Height, depth, node counts, diameter and the balanced check. All of them are one DFS that returns a number and updates an answer on the way up.

The naive balanced check is $O(n^2)$ because it recomputes heights. The bottom-up version returns the height and the verdict together, in one pass.

BOARD 1

Height And Depth

01 Height

1 + the taller child. An empty tree is 0. One pass, $O(n)$.

```
int height(Node *r) {
    if (!r) return 0;
    return 1 + max(height(r->left),
                   height(r->right));
}
```

CP The template for every bottom-up tree answer.

02 Min depth trap

A null child is not a leaf. Taking plain min returns 1 on a one-sided tree.

```
if (!r->left) return 1 + minDepth(r->right);
if (!r->right) return 1 + minDepth(r->left);
return 1 + min(minDepth(r->left),
               minDepth(r->right));
```

CP Handle the one-child case before the min.

03 Counting nodes

Same shape, add instead of max.

```
int count(Node *r) {
    if (!r) return 0;
    return 1 + count(r->left) + count(r->right);
}
```

CP Leaves: return 1 only when both children are null.

04 Depth of a node

Depth counts down from the root, height counts up from the leaves. Do not mix them.

```
// root depth 0, its children depth 1
// leaf height 1, root height h
```

CP Exams ask for one and students give the other.

REMEMBER

- ▶ Height = 1 + max of children.
- ▶ Empty tree height 0.
- ▶ Min depth: guard the one-child case.
- ▶ Depth counts down, height counts up.

BOARD 2

Diameter And Balanced

01 Naive balanced

Calling height at every node recomputes everything. $O(n^2)$.

```
bool bal(Node *r) {
    if (!r) return true;
    if (abs(height(r->left) - height(r->right)) > 1)
        return false;
    return bal(r->left) && bal(r->right);
}
```

CP Correct but too slow. Never the intended answer.

02 Bottom-up $O(n)$

Return the height, or -1 to mean unbalanced. One pass carries both facts.

```
int chk(Node *r) {
    if (!r) return 0;
    int l = chk(r->left), rr = chk(r->right);
    if (l < 0 || rr < 0 || abs(l - rr) > 1) return -1;
    return 1 + max(l, rr);
}
```

CP A sentinel return value is the whole trick.

03 Diameter

Longest path between any two nodes. It bends at some node: left height + right height.

```
int h(Node *r) {
    if (!r) return 0;
    int l = h(r->left), rr = h(r->right);
    best = max(best, l + rr);
    return 1 + max(l, rr);
}
```

CP Update the answer, then return the height.

04 The pattern

Compute the answer through the node, but return only what the parent needs.

```
// best = l + r      (answer here)
// return = 1 + max(l, r) (for the parent)
```

CP Same split appears in max path sum and tree DP.

REMEMBER

- ▶ Naive balanced check is $O(n^2)$.
- ▶ Diameter bends: $l + r$ at some node.
- ▶ Return -1 as the unbalanced sentinel.
- ▶ Update answer, return parent's value.

CHAPTER 7

2 BOARDS · 07_Tree_Views

Tree Views

What the tree looks like from outside. Every view here comes from one idea: give each node a horizontal distance, left is minus one and right is plus one.

Top and bottom view need a BFS so that ties break by level. Boundary traversal is the one with the awkward corner cases, so read that board twice.

BOARD 1

Top And Bottom View

01 Horizontal distance

Root is 0, left is hd-1, right is hd+1. One column per hd value.

```
// hd:  -2  -1   0   1   2
// each column seen from above or below
```

CP Every view problem is really a grouping by hd.

02 BFS with hd

Queue of {node, hd}. A map keyed by hd keeps the columns sorted automatically.

```
queue<pair<Node*, int>> q;
q.push({root, 0});
auto [n, hd] = q.front();
q.push({n->left, hd - 1});
q.push({n->right, hd + 1});
```

CP map<int,int> gives left-to-right order free.

03 Top view

Keep only the first node at each hd.

```
if (!m.count(hd)) m[hd] = n->val;
```

CP First in BFS order is the shallowest. Correct.

04 Bottom view

Overwrite every time, so the last one wins.

```
m[hd] = n->val;
```

CP DFS is wrong here: it can claim a column too early.

REMEMBER

- ▶ hd: left is -1, right is +1.
- ▶ Queue of {node, hd}, map<int,int> result.
- ▶ Top view keeps first, bottom keeps last.
- ▶ Must be BFS, not DFS.

BOARD 2

Vertical And Boundary

01 Vertical order

Group by hd, then by depth, then by value. All three keys matter for ties.

```
map<int, map<int, vector<int>>>> m;
m[hd][depth].push_back(n->val);
```

CP Nested maps sort both keys for you.

02 Reading it out

Outer loop is columns, inner loop is depth.

```
for (auto &[hd, col] : m)
    for (auto &[d, vals] : col)
        for (int v : vals) cout << v << " ";
```

CP Structured bindings keep this short.

03 Boundary traversal

Three pieces: left edge down, all leaves left to right, right edge up.

```
// 1. left boundary, excluding leaves
// 2. all leaves, left to right
// 3. right boundary, reversed
```

CP Collect the right edge, then reverse it.

04 Boundary corner cases

A leaf must not be counted twice. A single-node tree is just the root.

```
if (!r->left && !r->right) return; // skip in edges
```

CP Double-counted leaves are the usual bug.

REMEMBER

- ▶ Vertical: key by hd, then depth.
- ▶ map<int, map<int, vector<int>>>>.
- ▶ Boundary: left, leaves, right reversed.
- ▶ Never add a leaf twice.

MODULE THREE

Binary Trees: Path Sums, LCA and Ops

Trees you change rather than only read. Inverting and flattening, the path sum family, lowest common ancestor, and rebuilding a tree from its traversals.

8	Tree Structure Operations	2 boards
	Invert And Symmetric, Flatten To List	
9	Tree Path Sums	2 boards
	Root To Leaf, Max Path Sum	
10	LCA and Construction	3 boards
	LCA, Construct From Traversals, Serialization	

3 chapters · 7 boards · folders 08 to 10

CHAPTER 8

2 BOARDS · 08_Tree_Structure_Ops

Tree Structure Operations

Changing the tree rather than reading it. Invert, check symmetry, compare two trees, and flatten a tree into a right-leaning list.

Symmetric and identical look like the same function. They are not: one walks two subtrees in mirror order, the other walks them in the same order.

BOARD 1

Invert And Symmetric

01 Invert

Swap the children at every node. Post or pre order both work.

```
void invert(Node *r) {
    if (!r) return;
    swap(r->left, r->right);
    invert(r->left); invert(r->right);
}
```

CP Mirror image. $O(n)$, touches every node once.

02 Symmetric

Compare the tree against itself with two pointers moving outward in opposite ways.

```
bool sym(Node *a, Node *b) {
    if (!a || !b) return a == b;
    return a->val == b->val
        && sym(a->left, b->right)
        && sym(a->right, b->left);
}
```

CP left vs right and right vs left. Crossed.

03 Identical trees

Same idea but uncrossed.

```
return a->val == b->val
    && same(a->left, b->left)
    && same(a->right, b->right);
```

CP The only difference from symmetric is the crossing.

04 Subtree check

Try `same()` at every node of the big tree.

```
bool sub(Node *r, Node *t) {
    if (!r) return false;
    if (same(r, t)) return true;
    return sub(r->left, t) || sub(r->right, t);
}
```

CP $O(n*m)$ worst case. Usually accepted.

REMEMBER

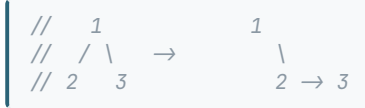
- ▶ Invert = swap children everywhere.
- ▶ Symmetric compares left/right crossed.
- ▶ Identical compares them straight.
- ▶ if (!a || !b) return a == b; handles nulls.

BOARD 2

Flatten To List

01 The goal

Right-skewed chain in preorder. Every left pointer ends up null.



CP The chain order is exactly preorder.

02 Reverse postorder

Walk right, then left, keeping the last node seen. Link backwards.

```

void flat(Node *r) {
    if (!r) return;
    flat(r->right); flat(r->left);
    r->right = prev; r->left = nullptr;
    prev = r;
}
  
```

CP Right before left, or you lose the subtree.

03 Morris style $O(1)$

Move the left subtree into the right chain, splicing it above the old right child.

```

while (cur) {
    if (!cur->left) { cur = cur->right; continue; }
    Node *p = cur->left;
    while (p->right) p = p->right;
    p->right = cur->right;
    cur->right = cur->left; cur->left = nullptr;
}
  
```

CP $O(1)$ space, no recursion, no stack.

04 Printing the chain

Follow right until null.

```

while (r) { cout << r->val << " "; r = r->right; }
  
```

CP Good sanity check that all lefts are null.

REMEMBER

- ▶ Result is a preorder right-chain.
- ▶ Recursive: go right, then left, link back.
- ▶ Morris: splice left subtree into right.
- ▶ All left pointers must end null.

CHAPTER 9

2 BOARDS · 09_Tree_Path_Sums

Tree Path Sums

The push and pop pattern. Add the node on the way down, recurse, then remove it before you return. Forget the pop and every answer after the first is wrong.

Maximum path sum is the one that bends. A node returns the best single downward arm, but the answer it updates is the best bend through itself.

BOARD 1

Root To Leaf

01 Has path sum

Subtract as you go down. At a leaf ask whether the remainder is exactly 0.

```
bool has(Node *r, int t) {
    if (!r) return false;
    if (!r->left && !r->right) return t == r->val;
    return has(r->left, t - r->val)
        || has(r->right, t - r->val);
}
```

CP Leaf test is both children null, not !r.

02 Push and pop

Add the node, recurse, then remove it. That undo is what makes it backtracking.

```
path.push_back(r->val);
if (!r->left && !r->right) all.push_back(path);
else { go(r->left); go(r->right); }
path.pop_back();
```

CP Forget the `pop_back` and every path is wrong.

03 All root to leaf paths

Collect at the leaf, print joined.

```
// 1→2→4
// 1→2→5→7
// 1→3→6→8
```

CP Same shape prints, sums, or counts them.

04 Root to leaf numbers

Build the number on the way down, add at leaves.

```
int go(Node *r, int cur) {
    if (!r) return 0;
    cur = cur * 10 + r->val;
    if (!r->left && !r->right) return cur;
    return go(r->left, cur) + go(r->right, cur);
}
```

CP Carry state down, combine on the way up.

REMEMBER

- ▶ Leaf = both children null.
- ▶ Subtract the target on the way down.
- ▶ `push_back`, recurse, `pop_back`.
- ▶ Missing `pop_back` is the classic bug.

BOARD 2

Max Path Sum

01 The path bends

Any path has a highest node. There it uses both children. Above it, only one.

```
// answer here = left + node + right
// give parent = node + max(left, right)
```

CP Two different values. This is the whole problem.

02 Drop negatives

A negative child contributes nothing. Clamp it to 0 instead of using it.

```
int l = max(0, go(r->left));
int rr = max(0, go(r->right));
```

CP Without the clamp, negative subtrees drag it down.

03 One pass

Update the global best, return the single branch.

```
int go(Node *r) {
    if (!r) return 0;
    int l = max(0, go(r->left));
    int rr = max(0, go(r->right));
    best = max(best, l + r->val + rr);
    return r->val + max(l, rr);
}
```

CP $O(n)$. Same shape as diameter.

04 All negative case

best must start at a very low value, not 0, or an all-negative tree answers 0.

```
int best = INT_MIN;
// tree -100(5(6,7)) → 18 via 6-5-7
```

CP Never initialise best to 0 here.

REMEMBER

- ▶ Answer bends: $l + \text{node} + r$.
- ▶ Clamp negative children to 0.
- ▶ Return only $\text{node} + \max(l, r)$.
- ▶ Start best at `INT_MIN`, not 0.

CHAPTER 10

3 BOARDS · 10_Tree_LCA_Construction

LCA and Construction

Lowest common ancestor in one DFS: if both sides come back non-null, you are standing on the answer.

Then the reverse direction. Preorder plus inorder, or postorder plus inorder, rebuilds the tree. Index the inorder into a map first or the build turns quadratic.

BOARD 1

LCA

01 The idea

Return the node where the two targets first end up on opposite sides.

```
Node *lca(Node *r, Node *p, Node *q) {
    if (!r || r == p || r == q) return r;
    Node *l = lca(r->left, p, q);
    Node *rr = lca(r->right, p, q);
    return (l && rr) ? r : (l ? l : rr);
}
```

CP Both sides non-null means this node is the split.

02 Reading the return

One side null: the answer is below the other. Both non-null: this node is the answer.

```
// l && rr → r is the LCA
// only l → answer is in the left subtree
```

CP $O(n)$, one pass, no parent pointers needed.

03 Path based

Record the root paths to both nodes, then walk forward while they agree.

```
// path to p: 1 2 5 7
// path to q: 1 2 4
// last match = 2
```

CP Easier to explain, but needs $O(h)$ extra space.

04 Distance via LCA

Depths from the root, minus twice the LCA depth.

```
d = dep(p) + dep(q) - 2 * dep(lca(root, p, q));
```

CP Standard follow-up question after LCA.

REMEMBER

- ▶ Base: !r or r is p or q, return r.
- ▶ Both sides non-null → r is the LCA.
- ▶ One side null → answer is on the other.
- ▶ $\text{dist} = d(p) + d(q) - 2 * d(\text{lca})$.

BOARD 2

Construct From Traversals

01 What you need

Inorder plus one of preorder or postorder. Pre plus post alone is not enough.

```
// pre  1 2 4 5 3
// in   4 2 5 1 3  → unique tree
```

CP Inorder is what splits left from right.

02 Pre plus in

Preorder front is the root. Find it in inorder: everything left of it is the left subtree.

```
int root = pre[preIdx++];
int mid = pos[root];
n->left  = build(inLo, mid - 1);
n->right = build(mid + 1, inHi);
```

CP Left must be built before right. Order matters.

03 Post plus in

Postorder back is the root. Walk it backwards and build right before left.

```
int root = post[postIdx--];
n->right = build(mid + 1, inHi);
n->left  = build(inLo, mid - 1);
```

CP Reversed postorder, so right subtree first.

04 Index map

Scanning inorder each time is $O(n^2)$. A map from value to index makes it $O(n)$.

```
map<int, int> pos;
for (int i = 0; i < n; i++) pos[in[i]] = i;
```

CP Assumes distinct values. Usually given.

REMEMBER

- ▶ Inorder plus pre or post gives a unique tree.
- ▶ Pre: root at front, build left first.
- ▶ Post: root at back, build right first.
- ▶ Value-to-index map makes it $O(n)$.

BOARD 3

Serialization

01 Why a null marker

Preorder alone is ambiguous. Writing the nulls makes the shape recoverable.

```
// tree 1(2, 3) → "1,2,#,#,3,#,#,"
```

CP One traversal plus null markers is enough.

02 Serialize

Preorder. Emit # for every null.

```
void ser(Node *r, string &s) {
    if (!r) { s += "#,"; return; }
    s += to_string(r->val) + ",";
    ser(r->left, s); ser(r->right, s);
}
```

CP Include the delimiter or 1 and 12 merge.

03 Deserialize

Read tokens in the same order. A # returns null and consumes one token.

```
Node *de(istringstream &in) {
    string t; getline(in, t, ',');
    if (t == "#") return nullptr;
    Node *n = new Node(stoi(t));
    n->left = de(in); n->right = de(in);
    return n;
}
```

CP The reader must consume in the same order.

04 Round trip

Serialize, deserialize, then compare traversals.

```
// in 1 2 3 → "1,2,#,#,3,#,#," → 1 2 3
```

CP Always verify by printing both traversals.

REMEMBER

- ▶ Preorder plus # for nulls.
- ▶ Deserialize reads in the same order.
- ▶ Keep the comma delimiter.
- ▶ Verify with a round-trip traversal.

MODULE FOUR

Binary Search Trees

One invariant buys you $O(h)$ search. This module is about using it, breaking it carefully during deletion, and putting it back.

11	BST Basics	2 boards
	Search Insert, Validation	

12	BST Operations	2 boards
	Deletion Successor, Kth Element Iterator	

13	BST Queries	2 boards
	Sorted Array To BST, Range Sum Two Sum	

3 chapters · 6 boards · folders 11 to 13

CHAPTER 11

2 BOARDS · 11_BST_Basics

BST Basics

One invariant: everything left is smaller, everything right is larger. Search and insert both walk one path, so both are $O(h)$.

$O(h)$ is not $O(\log n)$. Insert sorted data and the tree becomes a linked list.

BOARD 1

Search Insert

01 The invariant

Everything in the left subtree is smaller, everything right is larger. At every node.

```
//      8
//     / \
//    3   10    left < node < right
```

CP The invariant is subtree-wide, not just children.

02 Search $O(h)$

One comparison discards half the tree.

```
Node *find(Node *r, int x) {
    while (r && r->val != x)
        r = x < r->val ? r->left : r->right;
    return r;
}
```

CP Iterative form needs no stack. Prefer it.

03 Insert as a leaf

Walk down the same way, attach where you fall off the tree.

```
Node *ins(Node *r, int x) {
    if (!r) return new Node(x);
    if (x < r->val) r->left = ins(r->left, x);
    else r->right = ins(r->right, x);
    return r;
}
```

CP Reassigning the child link is what attaches it.

04 $O(h)$ is not $O(\log n)$

Sorted input makes a straight line. h becomes n and every op is $O(n)$.

```
// insert 1..7 in order → height 7
// balanced build      → height 3
```

CP $O(h)$. Only balanced trees give $O(\log n)$.

REMEMBER

- ▶ left < node < right, whole subtrees.
- ▶ Search and insert are $O(h)$.
- ▶ Insert always lands at a leaf.
- ▶ Sorted input degenerates to a list.

BOARD 2

Validation

01 The wrong check

Comparing only against the parent passes trees that are not BSTs.

```
//      10
//     /  \
//    5    15
//   /
//  6      6 < 10 but sits on the right
```

CP Local checks pass. This tree is not a BST.

02 Min max ranges

Carry an allowed interval down. Left tightens the max, right tightens the min.

```
bool ok(Node *r, long long lo, long long hi) {
    if (!r) return true;
    if (r->val <= lo || r->val >= hi) return false;
    return ok(r->left, lo, r->val)
        && ok(r->right, r->val, hi);
}
```

CP Use `long long` bounds to dodge INT_MIN/MAX.

03 Inorder must increase

A valid BST has a strictly increasing inorder. Keep only the previous value.

```
if (prev != nullptr && r->val <= *prev) return false;
prev = &r->val;
```

CP Strictly. Equal values break the invariant.

04 Which to use

Ranges read clearly, inorder needs $O(1)$ state. Both are $O(n)$.

```
// ranges : pass lo, hi down
// inorder: compare against prev only
```

CP Either is fine. Say strictly increasing out loud.

REMEMBER

- ▶ Parent-only comparison is wrong.
- ▶ Pass a (lo, hi) range down.
- ▶ Or check inorder is strictly increasing.
- ▶ `long long` bounds avoid INT_MIN/MAX traps.

CHAPTER 12

2 BOARDS · 12_BST_Operations

BST Operations

Deletion is the only fiddly operation in a BST, and only in the two-child case. Copy the inorder successor into the node, then delete the successor instead.

The kth smallest and the BST iterator are the same idea: an inorder walk you can pause. One counts, the other keeps a stack.

BOARD 1

Deletion Successor

01 Inorder successor

Smallest value greater than x. Go right once, then left as far as possible.

```
Node *ans = nullptr;
while (r) {
    if (x < r->val) { ans = r; r = r->left; }
    else r = r->right;
}
return ans;
```

CP Works even when x is not in the tree.

02 Delete: 0 or 1 child

Return the child, or null. The parent link is rewired by the return value.

```
if (!r->left) return r->right;
if (!r->right) return r->left;
```

CP These two lines cover leaf and one-child together.

03 Delete: 2 children

Copy in the successor's value, then delete the successor from the right subtree.

```
Node *s = r->right;
while (s->left) s = s->left;
r->val = s->val;
r->right = del(r->right, s->val);
```

CP Predecessor works too. Pick one and stay consistent.

04 Wiring it up

Always assign the recursive call back.

```
if (x < r->val) r->left = del(r->left, x);
else if (x > r->val) r->right = del(r->right, x);
else { /* the three cases */ }
return r;
```

CP Forgetting the assignment silently loses nodes.

REMEMBER

- ▶ Successor: right once, then all the way left.
- ▶ 0/1 child: return the other child.
- ▶ 2 children: copy successor, delete it.
- ▶ Always reassign the recursive result.

BOARD 2

Kth Element Iterator

01 Kth smallest

Inorder with a counter. Stop when the counter reaches k.

```
void go(Node *r) {
    if (!r || cnt >= k) return;
    go(r->left);
    if (++cnt == k) ans = r->val;
    go(r->right);
}
```

CP The `cnt >= k` guard must be in the base case too.

02 Kth largest

Reverse inorder: right, node, left.

```
go(r->right);
if (++cnt == k) ans = r->val;
go(r->left);
```

CP Same code, children swapped. Do not build a `vector`.

03 BST iterator

Push the whole left spine. `next()` pops, then pushes the left spine of its right child.

```
int next() {
    Node *n = st.top(); st.pop();
    Node *c = n->right;
    while (c) { st.push(c); c = c->left; }
    return n->val;
}
```

CP $O(h)$ space, amortised $O(1)$ per `next()`.

04 hasNext

Just whether the stack still has anything.

```
bool hasNext() { return !st.empty(); }
```

CP Beats flattening the tree when k is small.

REMEMBER

- ▶ Kth smallest: inorder plus a counter.
- ▶ Kth largest: reverse inorder.
- ▶ Iterator holds the left spine on a stack.
- ▶ $O(h)$ space, amortised $O(1)$ `next()`.

CHAPTER 13

2 BOARDS · 13_BST_Queries

BST Queries

Build a balanced BST from a sorted array by always taking the middle element. Inserting the same array one at a time gives you a chain instead.

Range queries use the invariant to prune. If the node is below the range there is no point looking left.

BOARD 1

Sorted Array To BST

01 Take the middle

The middle element becomes the root. Both halves get the same size, so it stays balanced.

```
int mid = (lo + hi) / 2;      // lo > hi → nullptr
Node *r = new Node(a[mid]);
r->left = build(a, lo, mid - 1);
r->right = build(a, mid + 1, hi);
return r;
```

CP $O(n)$. Height comes out as $\log n$.

02 Why it beats inserting

Inserting a sorted array one by one gives a straight line of height n .

```
// n = 7 sequential insert → height 7
// n = 7 mid recursion     → height 3
```

CP Same data, same tree type, very different height.

03 The other direction

Inorder gives the sorted array back.

```
void flat(Node *r, vector<int> &out) {
    if (!r) return;
    flat(r->left, out);
    out.push_back(r->val);
    flat(r->right, out);
}
```

CP Flatten, fix, rebuild is a common repair pattern.

04 Base case

$lo > hi$ means an empty range, so null.

```
if (lo > hi) return nullptr;
```

CP Using $lo \geq hi$ drops the last element.

REMEMBER

- ▶ Middle element becomes the root.
- ▶ Height comes out $\log n$, not n .
- ▶ Recurse on both halves. $O(n)$.
- ▶ Base case is $lo > hi$.

BOARD 2

Range Sum Two Sum

01 Range sum

Use the invariant to prune. A whole subtree can be skipped without looking inside.

```
int sum(Node *r, int lo, int hi) {
    if (!r) return 0;
    if (r->val < lo) return sum(r->right, lo, hi);
    if (r->val > hi) return sum(r->left, lo, hi);
    return r->val + sum(r->left, lo, hi)
        + sum(r->right, lo, hi);
}
```

CP Without pruning this is just a full traversal.

02 Counting in range

Same shape, add 1 instead of the value.

```
return 1 + cnt(r->left, lo, hi)
    + cnt(r->right, lo, hi);
```

CP Any range query on a BST follows this shape.

03 Two sum by flattening

Inorder gives a sorted array. Then the standard two-pointer walk.

```
int i = 0, j = n - 1;
while (i < j) {
    int s = a[i] + a[j];
    if (s == t) return true;
    s < t ? i++ : j--;
}
```

CP $O(n)$ time but $O(n)$ space. Fine most of the time.

04 Two sum in $O(h)$

Forward iterator from the smallest, reverse iterator from the largest. Move them inward.

```
// forward : left spine (ascending)
// backward : right spine (descending)
```

CP Same two-pointer logic, $O(h)$ space instead of $O(n)$.

REMEMBER

- ▶ Prune with the BST invariant.
- ▶ Two sum: flatten then two pointers.
- ▶ Below lo: go right. Above hi: go left.
- ▶ Two iterators give $O(h)$ space.

MODULE FIVE

Greedy Algorithms and Patterns

Greedy is easy to write and easy to get wrong. The work is in the sort order and the argument for why it is safe, not in the loop.

14	Greedy on Intervals	2 boards
	Exchange Argument, Merge And Activity Selection	

15	Greedy Scheduling	2 boards
	Min Platforms Sweep, Fractional Knapsack Job Sequencing	

16	Greedy on Arrays	2 boards
	Jump Game, Gas Station	

3 chapters · 6 boards · folders 14 to 16

CHAPTER 14

2 BOARDS · 14_Greedy_Intervals

Greedy on Intervals

A greedy solution needs an exchange argument: swapping any two items in your order never improves the answer. Without that argument you are guessing.

For interval problems the sort key is the whole solution. Merge by start, select by end, and be able to say why.

BOARD 1

Exchange Argument

01 What greedy needs

Sort by the right key, then show that swapping any adjacent pair never helps.

```
// if swapping neighbours cannot improve it,  
// the sorted order is optimal
```

CP This is the proof examiners want, not the code.

02 Shortest job first

Minimising total waiting time: run the shortest job first. Each job delays all the ones after it.

```
sort(d.begin(), d.end());  
long long wait = 0, total = 0;  
for (int x : d) { total += wait; wait += x; }
```

CP A long job early penalises everyone behind it.

03 Greedy is not universal

Change the objective and the same sort is wrong. Weighted completion needs value/time.

```
// by duration : 31  
// by w/t ratio: 23 ← optimal
```

CP Always check the key against the objective.

04 Verify small cases

Brute force every permutation for $n \leq 8$. If they disagree, your key is wrong.

```
sort(p.begin(), p.end());  
do { best = min(best, cost(p)); }  
while (next_permutation(p.begin(), p.end()));
```

CP Cheapest way to catch a wrong greedy.

REMEMBER

- ▶ Greedy = right sort key + exchange proof.
- ▶ New objective means a new sort key.
- ▶ Shortest first minimises total waiting.
- ▶ Brute force small n to check.

BOARD 2

Merge And Activity Selection

01 Merge overlapping

Sort by start. Extend the last interval when it overlaps, otherwise begin a new one.

```
sort(v.begin(), v.end());
for (auto [s, e] : v) {
    if (!out.empty() && s <= out.back().second)
        out.back().second = max(out.back().second, e);
    else out.push_back({s, e});
}
```

CP Take the max end. The next one may be shorter.

02 Activity selection

Maximum non-overlapping intervals. Sort by END, not by start.

```
sort(v.begin(), v.end(), [](auto a, auto b) {
    return a.second < b.second;
});
```

CP Earliest finish leaves the most room behind it.

03 Why end and not start

Sorting by start can pick one long interval and block everything after it.

```
// by end → 4 activities
// by start → 2 activities
```

CP The single most common greedy mistake.

04 The count loop

Take it if it starts at or after the last end.

```
int last = INT_MIN, cnt = 0;
for (auto [s, e] : v)
    if (s >= last) { cnt++; last = e; }
```

CP Removals to make it disjoint = $n - \text{cnt}$.

REMEMBER

- ▶ Merging: sort by start, take max end.
- ▶ Selection: sort by END.
- ▶ Earliest finish leaves the most room.
- ▶ Min removals = $n - \text{max kept}$.

CHAPTER 15

2 BOARDS · 15_Greedy_Scheduling

Greedy Scheduling

Counting overlaps by sweeping sorted events, and the two knapsack cousins where greedy is actually correct.

Fractional knapsack sorts by value per unit weight and works. The 0/1 version does not, and that difference is the point of the second board.

BOARD 1

Min Platforms Sweep

01 The question

Most trains present at any one instant. Pairing arrivals to departures is not needed.

```
// answer = peak concurrent count
```

CP Reframe 'how many rooms' as 'peak overlap'.

02 Two sorted arrays

Sort arrivals and departures separately. Advance whichever is smaller.

```
sort(arr.begin(), arr.end());
sort(dep.begin(), dep.end());
if (arr[i] <= dep[j]) { cur++; i++; }
else { cur--; j++; }
best = max(best, cur);
```

CP Sorting them apart is legal. You only need counts.

03 Event sweep

Same answer as +1 at arrival, -1 at departure, then scan in time order.

```
for (auto [t, d] : ev) { cur += d;
                        best = max(best, cur); }
```

CP Process -1 before +1 at equal times if touching is OK.

04 Min-heap version

Heap of end times. Pop everything that has finished, then push this one.

```
if (!pq.empty() && pq.top() <= s) pq.pop();
pq.push(e);
best = max(best, (int)pq.size());
```

CP Meeting rooms II. Heap size is the answer.

REMEMBER

- ▶ Answer = peak overlap, not pairings.
- ▶ Two-pointer over sorted arr and dep.
- ▶ Or a +1/-1 sweep line.
- ▶ Or a min-heap of end times.

BOARD 2

Fractional Knapsack Job Sequencing

01 Fractional knapsack

Sort by value/weight. Take whole items, then a fraction of the next one.

```
sort(it.begin(), it.end(), [](auto a, auto b) {
    return a.v * b.w > b.v * a.w;
});
```

CP Cross-multiply instead of dividing. No floats.

02 Filling the bag

Take all of it, or the part that fits.

```
if (w + it.w <= W) { w += it.w; val += it.v; }
else { val += it.v * (double)(W - w) / it.w;
      break; }
```

CP The break matters. The bag is full.

03 Why 0/1 is different

You cannot split an item, so the ratio order stops being optimal.

```
// ratio greedy on 0/1 : 9
// true optimum (DP)  : 10
```

CP 0/1 knapsack is DP. Never greedy.

04 Job sequencing

Sort by profit descending. Place each job in the latest free slot before its deadline.

```
for (int t = min(d, maxD); t >= 1; t--)
    if (!used[t]) { used[t] = true;
                    profit += p; break; }
```

CP Latest slot keeps the early ones open.

REMEMBER

- ▶ Fractional: sort by value/weight.
- ▶ Cross-multiply to avoid floating point.
- ▶ 0/1 knapsack is DP, not greedy.
- ▶ Job sequencing: latest free slot.

CHAPTER 16

2 BOARDS · 16_Greedy_Arrays

Greedy on Arrays

Two classic single-pass problems. Jump Game tracks the furthest reachable index. Gas Station resets the start whenever the running tank goes negative.

Both have an invariant you can state in one line. Find it before you write the loop.

BOARD 1

Jump Game

01 Can you reach the end

Track the furthest index reachable so far. If you stand past it, you are stuck.

```
int reach = 0;
for (int i = 0; i < n; i++) {
    if (i > reach) return false;
    reach = max(reach, i + a[i]);
}
return true;
```

CP $O(n)$, no DP needed. One variable.

02 Minimum jumps

Think in levels. A jump ends when you run out of the current level's range.

```
// level = every index reachable in k jumps
```

CP This is BFS on an array, written as a loop.

03 The level loop

Keep the current level end and the furthest reachable. Bump the count when you hit the end.

```
for (int i = 0; i < n - 1; i++) {
    far = max(far, i + a[i]);
    if (i == end) { jumps++; end = far; }
}
```

CP Loop to $n-1$. Landing on the last index is free.

04 When it is blocked

A zero can trap you. Detect it by end not moving forward.

```
if (i == end && far == end) return -1;
```

CP Only needed when the problem allows failure.

REMEMBER

- ▶ Reachability: track furthest reach.
- ▶ $i > \text{reach}$ means stuck.
- ▶ Min jumps: current level end + furthest.
- ▶ Loop to $n-1$, both are $O(n)$.

BOARD 2

Gas Station

01 Feasibility first

If total gas is less than total cost, no start works. Otherwise exactly one does.

```
if (total < 0) return -1;
```

CP Check the total before hunting for the index.

02 The reset rule

When the tank goes negative, no start inside that stretch can work. Restart after it.

```
if (tank < 0) { start = i + 1; tank = 0; }
```

CP One pass, $O(n)$. No need to retry each start.

03 Full loop

Two running sums: one to decide, one to test.

```
for (int i = 0; i < n; i++) {  
    int d = gas[i] - cost[i];  
    total += d; tank += d;  
    if (tank < 0) { start = i + 1; tank = 0; }  
}
```

CP total answers 'possible', tank answers 'where'.

04 The invariant

Every station skipped over was already proven to fail as a start.

```
// prefix sums from start were all  $\geq 0$   
// until i, so nothing inside can be better
```

CP State this and the proof is done.

REMEMBER

- ▶ total < 0 means impossible.
- ▶ Reset start to i+1 when tank < 0.
- ▶ One pass, $O(n)$, $O(1)$ space.
- ▶ Skipped stations are already ruled out.

MODULE SIX

Dynamic Programming: 1D, LIS and Grids

Same three questions every time: what is the state, what is the transition, what is the base case. Once those are written down the code is short.

17	DP Foundations	2 boards
	Memoisation vs Tabulation, Coin Change	

18	LIS and LCS	2 boards
	Longest Increasing Subsequence, Longest Common Subsequence	

19	Grids and Subset Sum	2 boards
	Grid Paths, Subset Sum	

3 chapters · 6 boards · folders 17 to 19

CHAPTER 17

2 BOARDS · 17_DP_Foundations

DP Foundations

Overlapping subproblems are the signal. Once you see the same call repeating, either memoise the recursion or fill a table bottom-up.

Coin change is the drill. Minimum coins, counting combinations and counting permutations are three different loop orders over the same state.

BOARD 1

Memoisation vs Tabulation

01 Overlapping subproblems

Plain recursion recomputes the same call thousands of times. That is the whole problem.

```
// fib(20) naive → 21891 calls
// fib(30) naive → 2692537 calls
```

CP If subproblems repeat, it is a DP.

02 Top-down memo

Same recursion, plus one array. Return early if the answer is already stored.

```
vector<long long> memo(n + 1, -1);
auto f = [&](auto &self, int i) -> long long {
    if (i <= 1) return i;
    if (memo[i] != -1) return memo[i];
    return memo[i] = self(self, i-1) + self(self, i-2);
};
```

CP Fastest way to write a DP under time pressure.

03 Bottom-up table

The same recurrence as a loop. Fill the base cases, then go forward.

```
vector<long long> dp(n + 1);
dp[0] = 0; dp[1] = 1;
for (int i = 2; i <= n; i++)
    dp[i] = dp[i-1] + dp[i-2];
```

CP No recursion depth risk. Prefer for large n.

04 Space reduction

If only the last two states are used, keep two variables.

```
long long a = 0, b = 1;
for (int i = 2; i <= n; i++) {
    long long c = a + b; a = b; b = c;
}
```

CP $O(1)$ space. Look at which rows you actually read.

REMEMBER

- ▶ Repeating subproblems means DP.
- ▶ Memo = recursion + cache array.
- ▶ Tabulation = same recurrence as a loop.
- ▶ Reduce space to the rows you read.

BOARD 2

Coin Change

01 Min coins state

$dp[a]$ = fewest coins to make amount a . Unreachable amounts stay at INF.

```
vector<int> dp(A + 1, INF);
dp[0] = 0;
for (int a = 1; a <= A; a++)
    for (int c : coins)
        if (c <= a && dp[a-c] != INF)
            dp[a] = min(dp[a], dp[a-c] + 1);
```

CP Guard the INF or a dead state looks cheap.

02 Counting combinations

Coin loop OUTSIDE. Each coin is considered once, so order does not matter.

```
for (int c : coins)
    for (int a = c; a <= A; a++)
        dp[a] += dp[a - c];    // 4 ways
```

CP Combinations: {1,2} and {2,1} counted once.

03 Counting permutations

Coin loop INSIDE. Every ordering is counted separately.

```
for (int a = 1; a <= A; a++)
    for (int c : coins)
        if (c <= a) dp[a] += dp[a - c];    // 9 ways
```

CP Same two loops, swapped. Very different answer.

04 Reconstruction

Store which coin was used, then walk back.

```
while (a > 0) { cout << take[a] << " ";
               a -= take[a]; }
```

CP Any 'which items' follow-up needs this array.

REMEMBER

- ▶ $dp[a]$ = best for amount a , $dp[0] = 0$.
- ▶ Always guard the INF sentinel.
- ▶ Coin loop outer = combinations.
- ▶ Coin loop inner = permutations.

CHAPTER 18

2 BOARDS · 18_DP_LIS_LCS

LIS and LCS

The $O(n^2)$ LIS is the honest starting point, and the $O(n \log n)$ version is worth memorising. Just remember the tails array is not the subsequence.

LCS is the two-string table every later string problem is built on, so learn to walk it backwards for the reconstruction.

BOARD 1

Longest Increasing Subsequence

01 The $O(n^2)$ state

$dp[i]$ = length of the LIS that ENDS at i . The answer is the max over all i .

```
vector<int> dp(n, 1);
for (int i = 0; i < n; i++)
    for (int j = 0; j < i; j++)
        if (a[j] < a[i])
            dp[i] = max(dp[i], dp[j] + 1);
```

CP 'Ends at i ' is what makes the transition work.

02 Reconstruction

Store the j that extended each i , then walk the parent chain backwards.

```
if (dp[j] + 1 > dp[i]) { dp[i] = dp[j] + 1;
                        par[i] = j; }
```

CP Reverse the collected chain at the end.

03 $O(n \log n)$ tails

$tails[k]$ = smallest possible tail of an increasing subsequence of length $k+1$.

```
for (int x : a) {
    auto it = lower_bound(tails.begin(),
                          tails.end(), x);
    if (it == tails.end()) tails.push_back(x);
    else *it = x;
}
```

CP Answer is $tails.size()$. Length only, not the sequence.

04 tails is not the LIS

It holds the best tails, not a real subsequence. Do not print it.

```
// a = 1 10 11 2
// tails = 1 2 11    but LIS = 1 10 11
```

CP Need the actual sequence? Use the parent array.

REMEMBER

- ▶ $dp[i]$ = LIS ending exactly at i .
- ▶ Answer is $\max(dp)$, not $dp[n-1]$.
- ▶ $O(n \log n)$ via `lower_bound` on tails.
- ▶ tails gives the length only.

BOARD 2

Longest Common Subsequence

01 The state

$dp[i][j]$ = LCS of the first i of s and the first j of t . Row and column 0 are 0.

```
if (s[i-1] == t[j-1])
    dp[i][j] = dp[i-1][j-1] + 1;
else
    dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
```

CP Match takes the diagonal, mismatch takes the best.

02 Reconstruction

Walk back from $dp[n][m]$. On a match, take the character and move diagonally.

```
while (i && j) {
    if (s[i-1] == t[j-1]) { res += s[i-1]; i--; j--; }
    else if (dp[i-1][j] > dp[i][j-1]) i--;
    else j--;
}
```

CP Reverse res at the end. It is built backwards.

03 Space reduction

Only the previous row is read, so two rows are enough. $O(\min(n, m))$ space.

```
vector<int> prev(m+1, 0), cur(m+1, 0);
// fill cur from prev, then swap
```

CP Kills reconstruction. Keep 2D if you need the string.

04 Common SUBSTRING

Contiguous, so a mismatch resets to 0 instead of inheriting.

```
if (s[i-1] == t[j-1])
    dp[i][j] = dp[i-1][j-1] + 1;
else dp[i][j] = 0; // reset
```

CP Substring resets, subsequence inherits. One line apart.

REMEMBER

- ▶ $dp[i][j]$ = LCS of prefixes i and j .
- ▶ Mismatch: max of left and up.
- ▶ Match: diagonal + 1.
- ▶ Substring resets to 0 on mismatch.

CHAPTER 19

2 BOARDS · 19_DP_Grids_Subsets

Grids and Subset Sum

Grid DP is DP with the state handed to you. Path counts, obstacles and minimum path sum are the same double loop with a different transition.

Subset sum introduces the 1D reduction. Loop the capacity downwards for 0/1 and upwards for unlimited copies. That single direction is the whole distinction.

BOARD 1

Grid Paths

01 Unique paths

$dp[i][j]$ = ways to reach cell (i,j) . First row and first column are all 1.

```
for (int i = 1; i < n; i++)
    for (int j = 1; j < m; j++)
        dp[i][j] = dp[i-1][j] + dp[i][j-1];
```

CP Only right and down moves. 3×7 grid gives 28.

02 Obstacles

A blocked cell has 0 ways. Set it and let the recurrence spread the zero.

```
if (g[i][j] == 1) dp[i][j] = 0;
else dp[i][j] = dp[i-1][j] + dp[i][j-1];
```

CP A blocked start or first row zeroes the rest of it.

03 Min path sum

Same table, min instead of sum, plus the cell's own cost.

```
dp[i][j] = g[i][j] + min(dp[i-1][j], dp[i][j-1]);
```

CP Max path sum is the same line with max.

04 One row of space

Only the row above is read, and $dp[i][j-1]$ is already the current row.

```
for (int i = 0; i < n; i++)
    for (int j = 1; j < m; j++)
        dp[j] += dp[j-1];
```

CP In place. $dp[j]$ is 'above', $dp[j-1]$ is 'left'.

REMEMBER

- ▶ $dp[i][j]$ = ways or best cost to (i,j) .
- ▶ Obstacles are just $dp = 0$.
- ▶ Base: first row and column.
- ▶ One rolling row gives $O(m)$ space.

BOARD 2

Subset Sum

01 The state

$dp[i][s]$ = can the first i items make sum s . $dp[0][0]$ is true, the rest of row 0 is false.

```
dp[i][s] = dp[i-1][s]           // skip
           || (s >= a[i-1]
              && dp[i-1][s - a[i-1]]); // take
```

CP Every subset DP is this take-or-skip pair.

02 1D reduction

One array, but the sum loop must go DOWN.

```
for (int x : a)
    for (int s = T; s >= x; s--)
        dp[s] = dp[s] || dp[s - x];
```

CP Descending so each item is used at most once.

03 Why downward

Going up reads a cell you already updated with this same item, so it reuses it.

```
// ascending: dp[2] then dp[4] with x=2
// → uses the 2 twice, wrong for 0/1
```

CP Loop direction is the whole 0/1 vs unbounded story.

04 The family

Same table answers several questions.

```
// feasible?      bool table
// how many?      int table, += instead of ||
// equal split?    target = total / 2
// min diff?      best reachable s ≤ total/2
```

CP Odd total means equal partition is impossible.

REMEMBER

- ▶ $dp[s]$ = is sum s reachable.
- ▶ take or skip, that is the transition.
- ▶ 1D: iterate s downward for 0/1.
- ▶ Equal partition needs an even total.

MODULE SEVEN

Dynamic Programming: Advanced and Partition

Knapsack and its relatives, the string tables, and the partition family where the state is an interval and you loop over split points.

20	Knapsack	2 boards
	0/1 Knapsack, Unbounded Knapsack & Rod Cutting	

21	String DP	2 boards
	Edit Distance, Shortest Common Supersequence	

22	Partition DP	2 boards
	Matrix Chain Multiplication, Partition DP	

3 chapters · 6 boards · folders 20 to 22

CHAPTER 20

2 BOARDS · 20_DP_Knapsack

Knapsack

Take it or leave it, on every item. That is the whole transition, and the 2D table collapses to one row once you see it.

Unbounded knapsack and rod cutting are the same code with the inner loop reversed.

BOARD 1

0/1 Knapsack

01 The state

$dp[i][w]$ = best value using the first i items within capacity w . Row 0 is all zeros.

```
// i items considered, w capacity left
// each item: take it or leave it
```

CP State it out loud before writing any loop.

02 The transition

Skip it, or take it and pay its weight.

```
dp[i][w] = dp[i-1][w];
if (w >= wt[i-1])
    dp[i][w] = max(dp[i][w],
                   val[i-1] + dp[i-1][w - wt[i-1]]);
```

CP Guard $w \geq wt$ or you index negative.

03 1D reduction

One array, capacity loop DOWNWARD so each item is used at most once.

```
for (int i = 0; i < n; i++)
    for (int w = W; w >= wt[i]; w--)
        dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
```

CP Reversed loop. Memorise this exact shape.

04 Which items

Walk the 2D table back. Value changed means the item was taken.

```
while (i > 0) {
    if (dp[i][w] != dp[i-1][w])
        { cout << i; w -= wt[i-1]; }
    i--;
}
```

CP Needs the full 2D table. The 1D version loses it.

REMEMBER

- ▶ $dp[i][w]$ = best value, first i , capacity w .
- ▶ Transition: skip or take.
- ▶ 1D 0/1: capacity loop descending.
- ▶ Item recovery needs the 2D table.

BOARD 2

Unbounded Knapsack & Rod Cutting

01 Unlimited copies

Same state as 0/1. Only the loop direction changes, and that allows reuse.

```
for (int i = 0; i < n; i++)
    for (int w = wt[i]; w <= W; w++)
        dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
```

CP Ascending. $dp[w - wt]$ may already include item i .

02 Direction is everything

Identical data, one loop flipped, two different answers.

```
// ascending (unbounded): 120
// descending (0/1)      : 110
```

CP If you remember one thing, remember this.

03 Rod cutting

Length is weight, price is value, pieces are unlimited. It is unbounded knapsack renamed.

```
for (int len = 1; len <= n; len++)
    for (int c = 1; c <= len; c++)
        dp[len] = max(dp[len], price[c] + dp[len - c]);
```

CP Recognise the disguise and you already know it.

04 Min count version

Swap max for min and add 1 per piece. Same ascending loop.

```
for (int c : coins)
    for (int a = c; a <= A; a++)
        dp[a] = min(dp[a], dp[a - c] + 1);
```

CP Min coins is unbounded knapsack with min.

REMEMBER

- ▶ Unbounded: capacity loop ASCENDING.
- ▶ 0/1: capacity loop DESCENDING.
- ▶ Rod cutting is unbounded knapsack.
- ▶ Same loops, max or min as needed.

CHAPTER 21

2 BOARDS · 21_DP_Strings

String DP

Edit distance is the three-move table: insert, delete, replace. Reading the moves back out of the table is a skill worth practising once.

Shortest common supersequence, minimum insertions and minimum deletions all come out of one LCS table.

BOARD 1

Edit Distance

01 The state

$dp[i][j]$ = edits to turn the first i of s into the first j of t .

```
dp[i][0] = i;      // delete all i
dp[0][j] = j;      // insert all j
```

CP The base row and column are the free answers.

02 The transition

Equal characters cost nothing. Otherwise pay 1 plus the best of three moves.

```
if (s[i-1] == t[j-1]) dp[i][j] = dp[i-1][j-1];
else dp[i][j] = 1 + min({dp[i-1][j-1], // replace
                        dp[i-1][j],    // delete
                        dp[i][j-1]});  // insert
```

CP Diagonal replace, up delete, left insert.

03 Reading the moves

Walk back from $dp[n][m]$ and name whichever neighbour you came from.

```
// replace h→r | delete r | delete e
```

CP Any 'show the operations' question needs this walk.

04 Space reduction

Only the previous row is read.

```
vector<int> prev(m+1), cur(m+1);
// prev[j] is dp[i-1][j], cur[j-1] is dp[i][j-1]
```

CP $O(m)$ space, but you lose the operation list.

REMEMBER

- ▶ $dp[i][j]$ = edits between prefixes i and j .
- ▶ Match: take the diagonal free.
- ▶ Base: $dp[i][0] = i$, $dp[0][j] = j$.
- ▶ Diagonal replace, up delete, left insert.

BOARD 2

Shortest Common Supersequence

01 Length from LCS

The shared part is written once. So the length is $n + m - \text{LCS}$.

```
int scs = n + m - lcs(s, t);
// "abac", "cab" : 4 + 3 - 2 = 5
```

CP One formula. Build the LCS table and you are done.

02 Building the string

Walk the LCS table back. On a match write once, otherwise write the one you step away from.

```
while (i && j) {
    if (s[i-1] == t[j-1]) { res += s[i-1]; i--; j--; }
    else if (dp[i-1][j] > dp[i][j-1]) res += s[--i];
    else res += t[--j];
}
```

CP Then drain both leftovers and reverse.

03 Min insert and delete

Same table. Whatever is not in the LCS must be removed or added.

```
int del = n - lcs;
int ins = m - lcs;
// total operations = del + ins
```

CP This is edit distance without replace.

04 One table, many answers

LCS, SCS, min insertions, min deletions all come from the same dp.

```
// lcs = dp[n][m]
// scs = n + m - lcs
// ops = (n - lcs) + (m - lcs)
```

CP Recognise them all as LCS in disguise.

REMEMBER

- ▶ SCS length = $n + m - \text{LCS}$.
- ▶ Deletions = $n - \text{LCS}$, insertions = $m - \text{LCS}$.
- ▶ Build it by walking the LCS table back.
- ▶ One LCS table answers all of these.

CHAPTER 22

2 BOARDS · 22_DP_Partition

Partition DP

The state is an interval and the loop is over split points. Matrix chain multiplication is the template.

Loop by increasing length, not by index, so the smaller intervals are already solved when you need them.

BOARD 1

Matrix Chain Multiplication

01 The dimension array

n matrices need $n+1$ dimensions. Matrix i is $d[i-1]$ by $d[i]$.

```
// d = {40, 20, 30, 10, 30}
// A1 40×20, A2 20×30, A3 30×10, A4 10×30
```

CP Off-by-one in the dimension array is the usual bug.

02 The state

$dp[i][j]$ = min cost to multiply $A_i..A_j$ into one matrix. $dp[i][i] = 0$.

```
// a single matrix costs nothing
```

CP Cost of one multiply is rows * mid * cols.

03 The split point

k is the LAST multiplication. Try every k and take the cheapest.

```
for (int k = i; k < j; k++)
    dp[i][j] = min(dp[i][j],
        dp[i][k] + dp[k+1][j]
        + d[i-1] * d[k] * d[j]);
```

CP The two sides are independent once k is fixed.

04 Loop by length

Shorter intervals must be filled first. Length is the outer loop, never i .

```
for (int len = 2; len <= n; len++)
    for (int i = 1; i + len - 1 <= n; i++) {
        int j = i + len - 1;
        /* try every k */
    }
```

CP $O(n^3)$. Wrong loop order reads empty cells.

REMEMBER

- ▶ n matrices, $n+1$ dimensions.
- ▶ $dp[i][j]$ = min cost for $A_i..A_j$.
- ▶ k is the last multiply: $dp[i][k] + dp[k+1][j]$.
- ▶ Outer loop is length, giving $O(n^3)$.

BOARD 2

Partition DP

01 Palindrome table first

`pal[i][j]` built by growing length, so `pal[i+1][j-1]` is already known.

```
pal[i][j] = (s[i] == s[j])
           && (len == 2 || pal[i+1][j-1]);
```

CP Precompute it. Checking inside the DP is too slow.

02 Min cuts

`dp[i]` = fewest cuts for the prefix `s[0..i]`. A palindromic prefix needs zero.

```
if (pal[0][i]) { dp[i] = 0; continue; }
for (int j = 1; j <= i; j++)
    if (pal[j][i])
        dp[i] = min(dp[i], dp[j-1] + 1);
```

CP "aab" gives 1: "aa" | "b".

03 Burst balloons

Ask which balloon is burst LAST in the interval. Then its neighbours are fixed.

```
dp[i][j] = max(dp[i][j], dp[i][k] + dp[k][j]
               + b[i] * b[k] * b[j]);
```

CP 'First' fails: the sides would still interact.

04 The generic template

Interval DP: loop by length, try every split, combine the two sides plus a join cost.

```
for (int len = 2; len <= n; len++)
    for (int i = 0; i + len <= n; i++)
        for (int k = i + 1; k < i + len; k++)
            /* combine */
```

CP MCM, cuts, balloons, stone merging: one shape.

REMEMBER

- ▶ Precompute the palindrome table.
- ▶ Balloons: pick the LAST one burst.
- ▶ `dp[i]` = min cuts for prefix `i`.
- ▶ Interval DP: length outer, split inner.

MODULE EIGHT

Graph Algorithms and Shortest Paths

Build the adjacency list, then pick the traversal that matches the weights. Four shortest path algorithms and two ways to build a minimum spanning tree.

23 Graph Traversals

3 boards

Representation, BFS and DFS, Cycle Detection, Bipartite Check and Topological Sort

24 Shortest Paths

2 boards

Dijkstra and 0-1 BFS, Bellman-Ford and Floyd-Warshall

25 Minimum Spanning Trees

2 boards

DSU and Kruskal, Prim

3 chapters · 7 boards · folders 23 to 25

CHAPTER 23

3 BOARDS · 23_Graph_Traversals

Graph Traversals

Adjacency list, then BFS or DFS. BFS gives shortest hop counts on an unweighted graph, which is the only shortest path you get for free.

Cycle detection is not one algorithm. Undirected graphs pass the parent down, directed graphs need a recursion stack. The two are not interchangeable.

BOARD 1

Representation, BFS and DFS

01 Adjacency list

$O(V+E)$ memory. The default choice. A matrix is $O(V^2)$, only fine for small V .

```
vector<vector<int>> g(n);
g[u].push_back(v);
g[v].push_back(u);      // undirected: both ways
```

CP Matrix only when $V \leq 1000$ or you need $O(1)$ lookup.

02 BFS gives shortest hops

On an unweighted graph, BFS distance is the minimum number of edges.

```
dist[s] = 0; q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : g[u]) if (dist[v] == -1)
        { dist[v] = dist[u] + 1; q.push(v); }
}
```

CP Mark visited when you PUSH, not when you pop.

03 DFS

Recursion, or a stack. Goes deep first.

```
auto dfs = [&](auto &self, int u) -> void {
    vis[u] = 1;
    for (int v : g[u]) if (!vis[v]) self(self, v);
};
dfs(dfs, 0);
```

CP Deep graphs can blow the stack. Use the iterative form.

04 Connected components

Loop over all nodes. Each unvisited one starts a new component.

```
for (int i = 0; i < n; i++)
    if (!vis[i]) { comp++; dfs(dfs, i); }
```

CP Never start from node 0 only. Graphs can be split.

REMEMBER

- ▶ Adjacency list, $O(V+E)$. Both directions.
- ▶ BFS = shortest hops, unweighted only.
- ▶ Mark visited on push, not on pop.
- ▶ Loop all nodes for components.

BOARD 2

Cycle Detection

01 Undirected: pass the parent

A visited neighbour is a cycle, unless it is the node you just came from.

```
bool dfs(int u, int p) {
    vis[u] = 1;
    for (int v : g[u]) {
        if (!vis[v]) { if (dfs(v, u)) return true; }
        else if (v != p) return true;
    }
    return false;
}
```

CP Without the parent check every edge looks like a cycle.

02 Undirected via BFS

Same rule, parent stored in an array.

```
if (!vis[v]) { par[v] = u; q.push(v); }
else if (v != par[u]) return true;
```

CP Use BFS if recursion depth is a worry.

03 Directed: recursion stack

A back edge to a node still ON the current path is a cycle. Visited alone is not enough.

```
col[u] = 1; // on the path
for (int v : g[u]) {
    if (col[v] == 1) return true;
    if (!col[v] && dfs(v)) return true;
}
col[u] = 2; // finished
```

CP Three colours: 0 new, 1 on path, 2 done.

04 Not interchangeable

A directed graph can reach a visited node legally. An undirected one cannot.

```
// 1 → 2, 1 → 3, 2 → 3    no cycle
// undirected version      has a cycle
```

CP Check whether the graph is directed first.

REMEMBER

- ▶ Undirected: skip the parent edge.
- ▶ Directed: needs a recursion-stack colour.
- ▶ 0 new, 1 on path, 2 finished.
- ▶ The two methods are not interchangeable.

BOARD 3

Bipartite Check and Topological Sort

01 Bipartite = 2-colouring

Colour neighbours the opposite colour. A clash means it is not bipartite.

```
col[s] = 0; q.push(s);
for (int v : g[u]) {
    if (col[v] == -1) { col[v] = 1 - col[u];
                        q.push(v); }
    else if (col[v] == col[u]) return false;
}
```

CP Not bipartite exactly when an odd cycle exists.

02 Kahn's algorithm

Repeatedly take a node with indegree 0. Directed acyclic graphs only.

```
for (int u : g) indeg[v]++;
for (int i = 0; i < n; i++)
    if (!indeg[i]) q.push(i);
```

CP Push every zero-indegree node, not just one.

03 The queue loop

Removing a node drops its neighbours' indegree.

```
while (!q.empty()) {
    int u = q.front(); q.pop();
    order.push_back(u);
    for (int v : g[u])
        if (--indeg[v] == 0) q.push(v);
}
```

CP `order.size() < n` means there is a cycle.

04 DFS topological sort

Push onto a stack after finishing a node. Reverse postorder is a valid order.

```
for (int v : g[u]) if (!vis[v]) dfs(v);
st.push(u);           // after all children
```

CP Kahn's also detects cycles. DFS needs colours for that.

REMEMBER

- ▶ Bipartite: BFS 2-colour, clash means no.
- ▶ Not bipartite = has an odd cycle.
- ▶ Kahn's: queue of indegree-0 nodes.
- ▶ Short output means a cycle exists.

CHAPTER 24

2 BOARDS · 24_Graph_Shortest_Paths

Shortest Paths

Dijkstra with a min-heap for non-negative weights, 0-1 BFS with a deque when every weight is zero or one.

Bellman-Ford is the one that survives negative edges and detects negative cycles. Floyd-Warshall is three loops for all pairs, and you must guard the infinities.

BOARD 1

Dijkstra and 0-1 BFS

01 Dijkstra needs non-negative

$O(E \log V)$. A negative edge breaks it: a settled node could still be improved.

```
priority_queue<pair<long long,int>,
    vector<pair<long long,int>>,
    greater<>> pq;
pq.push({0, s});
```

CP Negative weights mean Bellman-Ford, not Dijkstra.

02 The main loop

Pop the smallest, skip stale copies, relax every outgoing edge.

```
auto [d, u] = pq.top(); pq.pop();
if (d > dist[u]) continue; // stale
for (auto [v, w] : g[u])
    if (d + w < dist[v])
        { dist[v] = d + w; pq.push({dist[v], v}); }
```

CP The stale check is what keeps it $O(E \log V)$.

03 Path reconstruction

Record the parent on every improvement, then walk back and reverse.

```
par[v] = u;
// then: for (int c = t; c != -1; c = par[c])
```

CP Same trick as every other path DP.

04 0-1 BFS

Weights only 0 or 1: a deque replaces the heap. $O(V+E)$ instead of $O(E \log V)$.

```
if (w == 0) dq.push_front(v);
else dq.push_back(v);
```

CP 0 to the front keeps the deque sorted for free.

REMEMBER

- ▶ Dijkstra: non-negative weights only.
- ▶ Min-heap of {dist, node}, $O(E \log V)$.
- ▶ Skip stale entries: if ($d > \text{dist}[u]$).
- ▶ Weights 0/1: deque, push_front on 0.

BOARD 2

Bellman-Ford and Floyd-Warshall

01 Bellman-Ford

$O(VE)$. Handles negative weights. A shortest path uses at most $V-1$ edges.

```
for (int r = 0; r < n - 1; r++)
    for (auto [u, v, w] : edges)
        if (dist[u] != INF && dist[u] + w < dist[v])
            dist[v] = dist[u] + w;
```

CP $V-1$ rounds is exactly enough, never more.

02 Negative cycle test

If round V still improves something, no finite answer exists.

```
for (auto [u, v, w] : edges)
    if (dist[u] + w < dist[v]) return true;
```

CP Start dist at 0 everywhere to find any cycle.

03 Floyd-Warshall

All pairs, $O(V^3)$. k must be the OUTER loop.

```
for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

CP k inner is a classic wrong answer. Order matters.

04 Guard the INF

$INF + INF$ silently overflows and becomes a fake short path.

```
if (d[i][k] < INF && d[k][j] < INF)
    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

CP Or use a large-but-safe INF like $1e18$ on `long long`.

REMEMBER

- ▶ Bellman-Ford $O(VE)$, negatives allowed.
- ▶ Floyd-Warshall $O(V^3)$, k is OUTERMOST.
- ▶ $V-1$ rounds, then round V detects a cycle.
- ▶ Guard INF before adding.

CHAPTER 25

2 BOARDS · 25_Graph_MST

Minimum Spanning Trees

Kruskal sorts the edges and uses a disjoint set union to reject the ones that close a cycle. Prim grows a single tree with a heap.

Both give the same total weight. Pick Kruskal when the graph is sparse and you already have an edge list.

BOARD 1

DSU and Kruskal

01 DSU find

Path compression flattens the tree as it searches. Near $O(1)$ amortised.

```
int find(int x) {
    return p[x] == x ? x
        : p[x] = find(p[x]);
}
```

CP The `p[x]` = assignment IS the compression.

02 DSU union

Attach the smaller tree under the larger. Returns false if already joined.

```
bool join(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b);
    p[b] = a; sz[a] += sz[b];
    return true;
}
```

CP The false return is exactly the cycle test.

03 Kruskal

Sort all edges by weight. Take an edge unless it would close a cycle.

```
sort(e.begin(), e.end());
for (auto [w, u, v] : e)
    if (join(u, v)) { total += w; used++; }
```

CP $O(E \log E)$. Edge-driven, so good for sparse graphs.

04 Stopping condition

A spanning tree has exactly $V-1$ edges. Fewer means the graph is disconnected.

```
if (used != n - 1) cout << "not connected";
```

CP Always check. An MST may not exist.

REMEMBER

- ▶ find with path compression, union by size.
- ▶ join returns false = would make a cycle.
- ▶ Kruskal: sort edges, greedily add.
- ▶ MST has exactly $V-1$ edges.

BOARD 2

Prim

01 Grow one tree

Start anywhere. Repeatedly add the cheapest edge leaving the tree.

```
priority_queue<pair<int,int>,
    vector<pair<int,int>>,
    greater<>> pq;
pq.push({0, start});
```

CP Vertex-driven, unlike Kruskal's edge-driven view.

02 The loop

Pop the cheapest. Skip it if already in the tree, otherwise take the edge.

```
auto [w, u] = pq.top(); pq.pop();
if (inMST[u]) continue;
inMST[u] = true; total += w;
for (auto [v, wt] : g[u])
    if (!inMST[v]) pq.push({wt, v});
```

CP The inMST skip replaces Dijkstra's stale check.

03 Same total as Kruskal

Different edges are possible on ties, but the total weight is always equal.

```
// Kruskal total = 19
// Prim total    = 19
```

CP MST weight is unique. The edge set may not be.

04 Which to use

Prim is $O(E \log V)$ and stays connected. Kruskal needs a sort and a DSU.

```
// dense graph → Prim
// sparse graph → Kruskal
```

CP Already have the edge list? Kruskal is less code.

REMEMBER

- ▶ Prim grows one tree from a start vertex.
- ▶ Skip vertices already in the tree.
- ▶ Min-heap of {weight, vertex}.
- ▶ Same total weight as Kruskal, always.

MODULE NINE

Tree DP, Backtracking and Capstone

DP on a rooted tree, search with undo, and the two pointer patterns. The last chapter is also the one you should reach for first on array problems.

26	Tree DP	2 boards
	Subtree DP, Tree Diameter	

27	Backtracking	3 boards
	Subsets and Permutations, N-Queens, Word Search and Rat in a Maze	

28	Sliding Window and Two Pointers	2 boards
	Sliding Window, Two Pointers	

3 chapters · 7 boards · folders 26 to 28

CHAPTER 26

2 BOARDS · 26_Tree_DP

Tree DP

Root the tree, then let every node combine the answers its children return. Subtree sizes and sums are the warm-up.

Two-state tree DP is the pattern to keep: for each node, the best answer if you take it and the best answer if you do not.

BOARD 1

Subtree DP

01 Root it first

A tree has no cycles, so passing the parent replaces the visited array.

```
void dfs(int u, int p) {
    for (int v : g[u]) if (v != p) dfs(v, u);
}
```

CP No vis array needed. Just skip the parent.

02 Sizes and sums

Combine children on the way back up. One DFS gives every subtree at once.

```
sz[u] = 1; sum[u] = w[u];
for (int v : g[u]) if (v != p) {
    dfs(v, u);
    sz[u] += sz[v]; sum[u] += sum[v];
}
```

CP $O(n)$ for all n subtrees, not $O(n)$ each.

03 Two-state tree DP

`take[u]` = `u` is chosen. `skip[u]` = it is not. Choosing `u` forbids all its children.

```
take[u] = w[u] + sum of skip[v]
skip[u] = sum of max(take[v], skip[v])
```

CP Max independent set on a tree. Answer is max at the root.

04 The code

Accumulate both states in the same loop.

```
take[u] = w[u]; skip[u] = 0;
for (int v : g[u]) if (v != p) {
    dfs(v, u);
    take[u] += skip[v];
    skip[u] += max(take[v], skip[v]);
}
```

CP Every tree DP is 'what does the parent need to know'.

REMEMBER

- ▶ Pass the parent instead of a vis array.
- ▶ One DFS gives all subtree values.
- ▶ `take/skip` is the classic two-state pair.
- ▶ Taking `u` forces skipping its children.

BOARD 2

Tree Diameter

01 Where it bends

The longest path has a topmost node. There it joins the two deepest branches.

```
// answer at u = d1 + d2 (two deepest children)
// return to parent = 1 + d1
```

CP Same answer-vs-return split as max path sum.

02 One DFS

Track the two largest child depths.

```
int d1 = 0, d2 = 0;
for (int v : g[u]) if (v != p) {
    int d = dfs(v, u) + 1;
    if (d > d1) { d2 = d1; d1 = d; }
    else if (d > d2) d2 = d;
}
best = max(best, d1 + d2);
```

CP $O(n)$. Works on weighted trees too.

03 Two BFS method

BFS from anywhere to find the farthest node. BFS again from there. That distance is the diameter.

```
int a = farthest(0);
int b = farthest(a);
// dist(a, b) = diameter
```

CP Easy to remember and easy to explain.

04 Trees only

Two BFS relies on there being one path between any two nodes. Graphs break it.

```
// unweighted TREE → two BFS works
// general graph → it does not
```

CP Say 'tree' out loud before using the BFS trick.

REMEMBER

- ▶ Diameter bends: $d1 + d2$ at some node.
- ▶ Or: two BFS passes, farthest then farthest.
- ▶ Return $1 + d1$ to the parent.
- ▶ Two BFS is valid on trees only.

CHAPTER 27

3 BOARDS · 27_Backtracking

Backtracking

Choose, recurse, undo. The undo is the part people forget, and it is the reason the second answer comes out wrong.

Subsets, permutations, N-Queens and grid search are four faces of the same loop. N-Queens is where the $O(1)$ safety check earns its keep.

BOARD 1

Subsets and Permutations

01 Include or exclude

At each index, take it or skip it. 2^n subsets, one leaf each.

```
void go(int i) {
    if (i == n) { out.push_back(cur); return; }
    cur.push_back(a[i]); go(i + 1);
    cur.pop_back();      go(i + 1);
}
```

CP The `pop_back` is the backtrack. Never skip it.

02 Bitmask version

No recursion. Bit j set means take item j .

```
for (int m = 0; m < (1 << n); m++)
    for (int j = 0; j < n; j++)
        if (m >> j & 1) cout << a[j];
```

CP Cleaner when $n \leq 20$. Easy to loop over.

03 Skipping duplicates

Sort first, then at each level skip a value equal to the previous one.

```
for (int j = i; j < n; j++) {
    if (j > i && a[j] == a[j-1]) continue;
    /* take a[j], recurse, undo */
}
```

CP $j > i$, not $j > 0$. The first copy must be allowed.

04 Permutations by swapping

Swap into position i , recurse, swap back.

```
for (int j = i; j < n; j++) {
    swap(a[i], a[j]);
    go(i + 1);
    swap(a[i], a[j]);
}
```

CP $n!$ results. No visited array needed.

REMEMBER

- ▶ Subsets: take or skip, 2^n .
- ▶ Push, recurse, pop. Always undo.
- ▶ Duplicates: sort, then skip $j > i$ copies.
- ▶ Permutations: swap, recurse, swap back.

BOARD 2

N-Queens

01 One queen per row

Recurse on rows. Each level picks a column. That alone kills all row conflicts.

```
void go(int row) {
    if (row == n) { count++; return; }
    for (int c = 0; c < n; c++) /* try c */;
}
```

CP Rows are handled by the recursion structure itself.

02 Diagonal indexing

One diagonal has constant row+col. The other has constant row-col.

```
// d1 = row + col           range 0..2n-2
// d2 = row - col + n - 1   range 0..2n-2
```

CP The +n-1 shift keeps the index non-negative.

03 O(1) safety check

Three boolean arrays instead of scanning.

```
if (col[c] || d1[r+c] || d2[r-c+n-1]) continue;
col[c] = d1[r+c] = d2[r-c+n-1] = true;
go(r + 1);
col[c] = d1[r+c] = d2[r-c+n-1] = false;
```

CP Set, recurse, unset. The unset is the backtrack.

04 Solution counts

Worth knowing for sanity checks.

```
// n:  4  5  6  7  8
// ans: 2 10 4 40 92
```

CP n=6 giving 4 surprises people. It is correct.

REMEMBER

- ▶ Recurse by row, choose a column.
- ▶ Three bool arrays give O(1) checks.
- ▶ d1 = r+c, d2 = r-c+n-1.
- ▶ Set, recurse, unset.

BOARD 3

Word Search and Rat in a Maze

01 Grid DFS

Four directions. Stop on bounds or a mismatch.

```
int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};
```

CP Direction arrays beat four copy-pasted blocks.

02 Mark in place

Overwrite the cell, recurse, then restore it. No separate visited array needed.

```
char tmp = g[r][c];
g[r][c] = '#';
/* recurse in 4 directions */
g[r][c] = tmp;
```

CP The restore is what makes it backtracking, not DFS.

03 Why restore

Without it, one failed path permanently blocks cells that a different path needs.

```
// no restore → only the first route ever tried
// restore    → every route explored
```

CP A missing restore gives wrong answers, not slow ones.

04 Rat in a maze

Same shape, but record the moves and print the path when you reach the target.

```
path += 'D';
solve(r + 1, c);
path.pop_back();
```

CP The string push and pop mirrors the grid mark and restore.

REMEMBER

- ▶ Direction arrays for the four moves.
- ▶ Mark the cell, recurse, restore it.
- ▶ No restore means paths block each other.
- ▶ Path strings push and pop the same way.

CHAPTER 28

2 BOARDS · 28_Window_Two_Pointers

Sliding Window and Two Pointers

Two pointers that only move forward turn many $O(n^2)$ loops into $O(n)$. Fixed window, variable window, and the sorted-array pair scan.

Reach for this before DP on subarray and substring problems. Half the time it is enough.

BOARD 1

Sliding Window

01 Fixed size window

Add the new element, drop the one that fell out. $O(n)$, no inner loop.

```
for (int i = 0; i < n; i++) {
    sum += a[i];
    if (i >= k) sum -= a[i - k];
    if (i >= k - 1) best = max(best, sum);
}
```

CP Never recompute the window. Add one, remove one.

02 Variable window

Grow with right. Shrink from left while the window is invalid.

```
for (int r = 0; r < n; r++) {
    /* add a[r] */
    while (invalid) { /* remove a[l] */ l++; }
    best = max(best, r - l + 1);
}
```

CP This shape solves most window problems. Learn it cold.

03 Longest no repeat

Keep last-seen positions. Jump left past the previous copy.

```
if (last.count(s[r]) && last[s[r]] >= l)
    l = last[s[r]] + 1;
last[s[r]] = r;
```

CP The $\geq l$ check stops stale positions moving l backwards.

04 Smallest window

Same loop, but record the length while the window is still valid.

```
while (sum >= target) {
    best = min(best, r - l + 1);
    sum -= a[l++];
}
```

CP Minimising shrinks while valid. Maximising while invalid.

REMEMBER

- ▶ Fixed: add one, remove one. $O(n)$.
- ▶ Variable: grow right, shrink left.
- ▶ Maximise: shrink while invalid.
- ▶ Minimise: record while still valid.

BOARD 2

Two Pointers

01 Two sum, sorted

Ends move inward. Too small means move left up, too big means move right down.

```
int i = 0, j = n - 1;
while (i < j) {
    int s = a[i] + a[j];
    if (s == t) return true;
    s < t ? i++ : j--;
}
```

CP $O(n)$ after sorting. No hash map needed.

02 Three sum

Fix one index, two-pointer the rest. Skip duplicates at every level.

```
for (int i = 0; i < n - 2; i++) {
    if (i && a[i] == a[i-1]) continue;
    /* two pointers over i+1 .. n-1 */
}
```

CP $O(n^2)$. Skipping duplicates is most of the difficulty.

03 Slow and fast writer

One pointer reads, one writes. In-place filtering in $O(n)$.

```
int w = 1;
for (int r = 1; r < n; r++)
    if (a[r] != a[r-1]) a[w++] = a[r];
// w is the new length
```

CP Same idea as erase-remove, written by hand.

04 Picking the paradigm

Read the constraint before you choose.

```
// n ≤ 20      bitmask or backtracking
// n ≤ 500      $O(n^3)$  interval DP
// n ≤ 1e5      $O(n \log n)$  sort or window
// n ≤ 1e6      $O(n)$  only
```

CP The constraint tells you the intended complexity.

REMEMBER

- ▶ Sorted two sum: ends move inward.
- ▶ Three sum: fix one, two-pointer the rest.
- ▶ Read/write pointers filter in place.
- ▶ Constraints reveal the intended complexity.